Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Design and Analysis of Algorithm (CSDS103)
Lab Assignment: Comprehensive Algorithmic Analysis
Lab Assignment 9: Backtracking Algorithms

Assignment 1: Optimal License Purchase Order (Greedy Algorithm)

Problem Statement

Suppose a consulting firm that offers security consultancy needs to obtain licenses for n different pieces of cryptographic software. Due to regulations, they can buy only one license per month. Each license currently costs Rs 1000, but the cost of each license increases at an exponential rate $r_j (> 1)$ every month. For example, if license j is purchased t months from now, it will cost $1000 \times r_j^t$. All price growth rates are distinct. Given the rates of price growth, determine the order in which licenses should be bought to minimize total cost. Implement this in a program.

Algorithm Design & Proof of Correctness

- **Objective**: Find a permutation π of licenses $\{1, 2, \ldots, n\}$ minimizing the total purchase cost:

$$\text{Total Cost} = \sum_{t=0}^{n-1} 1000 \cdot r_{\pi(t)}^t$$

- **Exchange Argument**: Suppose in an optimal schedule π, a license A with rate r_A is purchased at month t, and license B with rate r_B is purchased at month $t + 1$, such that $r_A < r_B$. Swapping their positions changes the cost contribution of these two items from $1000(r_A^t + r_B^{t+1})$ to $1000(r_B^t + r_A^{t+1})$. Taking the difference:

$$\Delta = 1000(r_B^t + r_A^{t+1}) - 1000(r_A^t + r_B^{t+1}) = 1000\left[r_A^t(r_A - 1) - r_B^t(r_B - 1)\right]$$

  Since $r_B > r_A > 1$, we have $r_B^t > r_A^t \geq 1$ and $(r_B - 1) > (r_A - 1) > 0$. Thus $r_A^t(r_A - 1) < r_B^t(r_B - 1)$, meaning $\Delta < 0$. Therefore, purchasing B before A strictly reduces the cost.

- **Greedy Strategy**: Sort all licenses in strictly **descending order** of growth rate r_j.

- **Time Complexity**: $O(n \log n)$ due to comparison-based sorting; Space Complexity is $O(n)$.

Python Implementation

```python
def min_cost_license_order(licenses, base_cost=1000):
    """
    Determines optimal purchase order to minimize total license cost.
    licenses: list of (license_name, rate r_j) where r_j > 1
    """
    # Sort in descending order of price growth rate r_j
    sorted_licenses = sorted(licenses, key=lambda x: x[1], reverse=True)

    schedule = []
    total_cost = 0.0
    for month, (name, r) in enumerate(sorted_licenses):
        cost = base_cost * (r ** month)
        schedule.append({"month": month, "name": name, "rate": r, "cost": cost})
        total_cost += cost

    return schedule, total_cost

# Demonstration for Assignment 1
licenses = [
    ("CryptoTool Alpha", 1.25),
    ("CryptoTool Beta",  1.80),
    ("CryptoTool Gamma", 1.10),
    ("CryptoTool Delta", 1.45)
]

schedule, total_cost = min_cost_license_order(licenses, base_cost=1000)

print("Optimal License Purchase Schedule:")
print("-" * 65)
print(f"{'Month':<7} | {'Software':<18} | {'Growth Rate (r_j)':<18} | {'Cost (Rs)':<10}")
print("-" * 65)
for item in schedule:
    print(f"{item['month']:<7} | {item['name']:<18} | {item['rate']:<18.2f} | Rs {item['cost']:<10.2f}")
print("-" * 65)
print(f"Total Minimized Cost: Rs {total_cost:.2f}")
```

Program Execution Output

```
Optimal License Purchase Schedule:
-----------------------------------------------------------------
Month   | Software           | Growth Rate (r_j)  | Cost (Rs)
-----------------------------------------------------------------
0       | CryptoTool Beta    | 1.80               | Rs 1000.00
1       | CryptoTool Delta   | 1.45               | Rs 1450.00
2       | CryptoTool Alpha   | 1.25               | Rs 1562.50
3       | CryptoTool Gamma   | 1.10               | Rs 1331.00
-----------------------------------------------------------------
Total Minimized Cost: Rs 5343.50
```

Assignment 2: Directed Graph Euler Circuit (Hierholzer's Algorithm)

Problem Statement

An Euler circuit in a directed graph is a cycle in which every edge is visited exactly once. Design and implement a linear-time algorithm to find an Euler circuit in a directed graph (where one exists).

Algorithm Design & Theoretical Conditions

A directed graph $G = (V, E)$ possesses an Euler circuit if and only if:

1. **Balanced Degree Condition**: Every vertex $v \in V$ satisfies $\text{in-degree}(v) = \text{out-degree}(v)$.
2. **Connectedness Condition**: All vertices with non-zero degree belong to a single connected component.

Hierholzer's Algorithm (Linear Time $O(V + E)$):

1. Verify that $\text{in-degree}(v) = \text{out-degree}(v)$ for all $v \in V$. If not, no Euler circuit exists.
2. Start traversal from any vertex s with $\text{out-degree}(s) > 0$.
3. Maintain a traversal stack. At vertex u, if unused outgoing edges exist, pop the edge (u, v) and push v to the stack.
4. When the current vertex has no remaining outgoing edges, pop it from the stack and append it to the circuit list.
5. Reverse the resulting circuit list. Verify the circuit length equals $|E| + 1$ to confirm full edge coverage.

Python Implementation

```python
import collections

def find_euler_circuit_directed(edges):
    """
    Finds an Euler circuit in a directed graph in linear time O(V + E).
    edges: list of (u, v) representing directed edge u -> v.
    Returns: list of vertices forming the Euler circuit, or None if none exists.
    """
    if not edges:
        return []

    in_degree = collections.defaultdict(int)
    out_degree = collections.defaultdict(int)
    adj = collections.defaultdict(list)

    for u, v in edges:
```

```
        out_degree[u] += 1
        in_degree[v] += 1
        adj[u].append(v)

    # 1. Check in-degree == out-degree for every vertex
    all_vertices = set(in_degree.keys()) | set(out_degree.keys())
    for v in all_vertices:
        if in_degree[v] != out_degree[v]:
            return None  # Degree condition violated

    # 2. Hierholzer's Algorithm using a stack
    start_vertex = edges[0][0]
    stack = [start_vertex]
    circuit = []

    # Copy adjacency lists so we can pop edges in O(1)
    edge_map = {u: list(neighbors) for u, neighbors in adj.items()}

    while stack:
        curr = stack[-1]
        if edge_map.get(curr):
            next_vertex = edge_map[curr].pop()
            stack.append(next_vertex)
        else:
            circuit.append(stack.pop())

    circuit.reverse()

    # 3. Verify all edges were visited (connectivity check)
    if len(circuit) != len(edges) + 1:
        return None

    return circuit

# Demonstration for Assignment 2
# Graph with Euler circuit: 0->1->2->0 and 0->3->4->0
graph_edges = [
    (0, 1), (1, 2), (2, 0),
    (0, 3), (3, 4), (4, 0)
]

circuit = find_euler_circuit_directed(graph_edges)
if circuit:
    path_str = " -> ".join(map(str, circuit))
    print("Euler Circuit Found:")
    print(path_str)
    print(f"Total edges in graph: {len(graph_edges)}, Total traversed edges: {len(
    circuit)-1}")
else:
    print("No Euler circuit exists for this graph.")

# Testing a non-Eulerian directed graph
non_eulerian_edges = [(0, 1), (1, 2), (2, 3)]
print("\nNon-Eulerian graph check result:", find_euler_circuit_directed(
    non_eulerian_edges))
```

Program Execution Output

```
Euler Circuit Found:
0 -> 3 -> 4 -> 0 -> 1 -> 2 -> 0
Total edges in graph: 6, Total traversed edges: 6

Non-Eulerian graph check result: None
```

Assignment 3: Pick Up Sticks (3D Geometry & Topological Sort)

Problem Statement

You are given a set of N sticks, each specified by its two endpoints as ordered triples (x, y, z). No stick is vertical, and a stick can only be picked up if no other sticks are lying on top of it.

(a) Write a routine that takes two sticks a and b and reports whether a is `above`, `below`, or `unrelated` to b.

(b) Give an algorithm that determines whether it is possible to pick up all the sticks, and if so, provides a sequence of stick pickups that accomplishes this.

Algorithm Design

- **Part (a) Relative Position Routine**: Since no stick is vertical, their projections onto the xy-plane are non-degenerate 2D line segments:

$$P_a(t) = P_1 + t(P_2 - P_1), \quad t \in [0, 1]$$

$$P_b(u) = Q_1 + u(Q_2 - Q_1), \quad u \in [0, 1]$$

  We compute the 2D intersection parameters (t, u) via 2D determinant Cramer's rule. If $0 \leq t, u \leq 1$, the segments cross in 2D. We then evaluate their z-coordinates at the crossing point:

$$z_a = z_1 + t(z_2 - z_1), \quad z_b = z_3 + u(z_4 - z_3)$$

  If $z_a > z_b$, a is `above` b; if $z_a < z_b$, a is `below` b. If they do not intersect in 2D, they are `unrelated`.

- **Part (b) Pickup Order via Topological Sort**: If stick a is above stick b, stick a physically rests on top of stick b, meaning a must be removed prior to b. We construct a directed dependency graph $G = (V, E)$ where a directed edge $a \rightarrow b$ signifies that stick a must be picked up before stick b. A stick can be picked up when its in-degree is 0. We execute Kahn's Topological Sort algorithm ($O(N^2)$). If the topological sequence contains all N sticks, pickup is possible; otherwise, a cycle of mutual obstruction exists.

Python Implementation

```python
import collections

def relative_position(stick_a, stick_b):
    """
    Reports whether stick_a is 'above', 'below', or 'unrelated' to stick_b.
    Each stick is specified as ((x1, y1, z1), (x2, y2, z2)).
    """
    (x1, y1, z1), (x2, y2, z2) = stick_a
    (x3, y3, z3), (x4, y4, z4) = stick_b

    dx_a, dy_a = x2 - x1, y2 - y1
    dx_b, dy_b = x4 - x3, y4 - y3

    det = dx_a * dy_b - dy_a * dx_b

    # Parallel or collinear in 2D projection
    if abs(det) < 1e-9:
        return "unrelated"

    # Solve for intersection parameters t and u
    t = ((x3 - x1) * dy_b - (y3 - y1) * dx_b) / det
    u = ((x3 - x1) * dy_a - (y3 - y1) * dx_a) / det

    # Check if the intersection point lies on both line segments
    eps = 1e-9
    if not (-eps <= t <= 1 + eps and -eps <= u <= 1 + eps):
        return "unrelated"

    # Interpolate z-heights at the intersection point
    z_a = z1 + t * (z2 - z1)
    z_b = z3 + u * (z4 - z3)

    if z_a > z_b + eps:
        return "above"
    elif z_b > z_a + eps:
        return "below"
    else:
        return "unrelated"


def pick_up_sticks(sticks):
    """
    Determines if all sticks can be picked up, and returns a valid pickup sequence.
    """
    n = len(sticks)
    adj = collections.defaultdict(list)
    in_degree = [0] * n

    # Directed edge: a -> b means 'a' is above 'b' (a must be picked up before b)
    for i in range(n):
        for j in range(n):
            if i != j:
                if relative_position(sticks[i], sticks[j]) == "above":
                    adj[i].append(j)
                    in_degree[j] += 1

```

```
    # Topological sorting (Kahn's Algorithm)
    queue = collections.deque([i for i in range(n) if in_degree[i] == 0])
    pickup_sequence = []

    while queue:
        curr = queue.popleft()
        pickup_sequence.append(curr)
        for neighbor in adj[curr]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(pickup_sequence) == n:
        return True, pickup_sequence
    else:
        return False, []

# Demonstration for Assignment 3
# Define 3 sticks:
# Stick 0 lies horizontally at z=5 across (0,0) to (10,10)
# Stick 1 lies across (0,10) to (10,0) at z=2 (below stick 0)
# Stick 2 lies isolated at (20,20) to (30,30) at z=1
stick0 = ((0, 0, 5), (10, 10, 5))
stick1 = ((0, 10, 2), (10, 0, 2))
stick2 = ((20, 20, 1), (30, 30, 1))

sticks = [stick0, stick1, stick2]

print("Pairwise relations:")
print("Stick 0 vs Stick 1:", relative_position(stick0, stick1))
print("Stick 1 vs Stick 0:", relative_position(stick1, stick0))
print("Stick 0 vs Stick 2:", relative_position(stick0, stick2))

possible, sequence = pick_up_sticks(sticks)
if possible:
    print("\nPickup feasible! Sequence of stick indices:", sequence)
else:
    print("\nCannot pick up all sticks (mutual obstruction cycle detected).")
```

Program Execution Output

```
Pairwise relations:
Stick 0 vs Stick 1: above
Stick 1 vs Stick 0: below
Stick 0 vs Stick 2: unrelated

Pickup feasible! Sequence of stick indices: [0, 2, 1]
```

Assignment 4: Graph Two-Colorability (Bipartiteness) & Applications

Problem Statement

A graph is k-colorable if each vertex can be given one of k colours and no edge connects identically coloured vertices. Give a linear-time algorithm to test a graph for two-colorability. Assume graphs are stored in adjacency-list format and specify any additional data structures that are needed. How is a two-colorable graph related to a bipartite graph? Provide at least two applications of two-colorability or bipartite graphs.

Theoretical Analysis & Data Structures

1. **Data Structures Required**:
   - **Adjacency List**: To store the input graph where each vertex maps to its list of neighbors.
   - **Color Map / Array** (`colors`): Maps each vertex $v \in V$ to color `0`, `1`, or `unvisited` in $O(1)$ time.
   - **Queue** (`collections.deque`): FIFO queue for standard Breadth-First Search (BFS) component traversal.
2. **Relationship between Two-Colorable and Bipartite Graphs**: A graph is **two-colorable if and only if it is bipartite**:
   - In a 2-colorable graph, let $V_1 = \{v \in V \mid \text{color}(v) = 0\}$ and $V_2 = \{v \in V \mid \text{color}(v) = 1\}$. Since no two adjacent vertices share the same color, every edge $(u, v) \in E$ has one endpoint in V_1 and the other in V_2, satisfying the definition of a bipartite graph.
   - Fundamental Graph Theory Theorem: A graph is bipartite (and thus 2-colorable) if and only if it contains **no odd-length cycle**.
3. **Two Real-World Applications**:
   (a) **Matching & Assignment Problems**: Bipartite graphs model two disjoint sets of entities (e.g., job applicants and positions, medical residents and hospitals, tasks and compute nodes). Maximum matching algorithms resolve allocation problems.
   (b) **Channel / Frequency Allocation in Conflict Networks**: Determining whether a network of communication towers or tasks can operate collision-free using only two distinct frequency bands or time slots without mutual interference.

Python Implementation

```
def test_two_colorability(adj_list):
    """
    Tests graph two-colorability in linear time O(V + E) using BFS.
    adj_list: dict mapping vertex -> list of neighbor vertices.
    Returns: (is_two_colorable, color_map)
```

```
    """
    colors = {}

    for start_node in adj_list:
        if start_node not in colors:
            colors[start_node] = 0
            queue = collections.deque([start_node])

            while queue:
                u = queue.popleft()
                current_color = colors[u]

                for neighbor in adj_list.get(u, []):
                    if neighbor not in colors:
                        # Assign opposite color
                        colors[neighbor] = 1 - current_color
                        queue.append(neighbor)
                    elif colors[neighbor] == current_color:
                        # Found edge between identically colored vertices -> odd cycle
    !
                        return False, None

    return True, colors

# Demonstration for Assignment 4
# Example 1: Bipartite graph (Square C4 cycle)
graph_bipartite = {
    'A': ['B', 'D'],
    'B': ['A', 'C'],
    'C': ['B', 'D'],
    'D': ['A', 'C']
}

# Example 2: Non-bipartite graph (Triangle C3 odd cycle)
graph_odd_cycle = {
    'X': ['Y', 'Z'],
    'Y': ['X', 'Z'],
    'Z': ['X', 'Y']
}

bip_status, bip_colors = test_two_colorability(graph_bipartite)
print("Graph 1 (Square C4):")
print(f"  Is Two-Colorable (Bipartite)? {bip_status}")
print(f"  Color Assignment: {bip_colors}")

odd_status, _ = test_two_colorability(graph_odd_cycle)
print("\nGraph 2 (Triangle C3):")
print(f"  Is Two-Colorable (Bipartite)? {odd_status} (Contains odd cycle, cannot 2-
    color)")
```

Program Execution Output

```
Graph 1 (Square C4):
  Is Two-Colorable (Bipartite)? True
  Color Assignment: {'A': 0, 'B': 1, 'D': 1, 'C': 0}
```

```
Graph 2 (Triangle C3):
  Is Two-Colorable (Bipartite)? False (Contains odd cycle, cannot 2-color)
```

Assignment 5: Maximum Bipartite Matching via Network Flow

Problem Statement

Edge-weighted graphs can be interpreted as a network of pipes, where the weight of edge (i, j) determines the pipe's capacity. The network flow problem asks for the maximum flow from a source vertex s to a target vertex t in a weighted graph G, respecting the capacity of each pipe.

- Create a source node s connected to every vertex in set L by an edge of weight 1.
- Create a sink node t and connect it to every vertex in set R by an edge of weight 1.
- Assign each edge in the bipartite graph G a weight of 1.

Now, the maximum possible flow from s to t defines the largest matching in G. Implement the algorithm.

Flow Network Formulation

Given bipartite graph $G = (L \cup R, E)$:

1. Construct directed flow network $G' = (V', E')$ where $V' = L \cup R \cup \{s, t\}$.
2. Connect source $s \to u$ for all $u \in L$ with capacity $c(s, u) = 1$.
3. Connect all $v \in R \to t$ with capacity $c(v, t) = 1$.
4. Direct all bipartite edges $u \to v$ for $(u, v) \in E$ with capacity $c(u, v) = 1$.
5. By the Integrality Theorem, the maximum flow value equals the maximum cardinality matching, and edges (u, v) carrying flow $f(u, v) = 1$ constitute the optimal matching pairs.
6. Solved using Edmonds-Karp algorithm (BFS augmenting paths) in $O(|V| \cdot |E|)$ on unit networks.

Python Implementation

```
def max_bipartite_matching_flow(set_L, set_R, bipartite_edges):
    """
    Computes Maximum Bipartite Matching using Edmonds-Karp Network Flow.
    """
```

```
    source = "source_s"
    sink = "sink_t"

    capacity = collections.defaultdict(lambda: collections.defaultdict(int))
    adj = collections.defaultdict(set)

    def add_edge(u, v, cap):
        capacity[u][v] = cap
        adj[u].add(v)
        adj[v].add(u)

    # 1. Connect source s to every vertex in L with weight 1
    for u in set_L:
        add_edge(source, u, 1)

    # 2. Connect every vertex in R to sink t with weight 1
    for v in set_R:
        add_edge(v, sink, 1)

    # 3. Assign each edge in bipartite graph G weight 1
    for u, v in bipartite_edges:
        add_edge(u, v, 1)

    # Edmonds-Karp algorithm (BFS for shortest augmenting path)
    max_flow = 0
    while True:
        parent = {}
        queue = collections.deque([source])
        while queue and sink not in parent:
            curr = queue.popleft()
            for neighbor in adj[curr]:
                if neighbor not in parent and capacity[curr][neighbor] > 0:
                    parent[neighbor] = curr
                    queue.append(neighbor)

        # No more augmenting paths found
        if sink not in parent:
            break

        # Augment flow along path
        v = sink
        while v != source:
            u = parent[v]
            capacity[u][v] -= 1
            capacity[v][u] += 1
            v = u
        max_flow += 1

    # Extract matching: edges u -> v where flow is 1 (residual capacity is 0)
    matching = []
    for u in set_L:
        for v in set_R:
            if capacity[u][v] == 0 and capacity[v][u] == 1:
                matching.append((u, v))

    return max_flow, matching

# Demonstration for Assignment 5
```

```python
# Left partition: Applicants (L1, L2, L3, L4)
# Right partition: Jobs (J1, J2, J3, J4)
L = ["Applicant_1", "Applicant_2", "Applicant_3", "Applicant_4"]
R = ["Job_1", "Job_2", "Job_3", "Job_4"]

# Potential matches (edges in G)
bipartite_edges = [
    ("Applicant_1", "Job_1"),
    ("Applicant_1", "Job_2"),
    ("Applicant_2", "Job_2"),
    ("Applicant_3", "Job_2"),
    ("Applicant_3", "Job_3"),
    ("Applicant_4", "Job_3"),
    ("Applicant_4", "Job_4")
]

max_matching_size, matching_pairs = max_bipartite_matching_flow(L, R, bipartite_edges)

print(f"Maximum Bipartite Matching Size (Max Flow): {max_matching_size}\n")
print("Matched Pairs (Applicant -> Job):")
for u, v in matching_pairs:
    print(f"  {u} <===> {v}")
```

Program Execution Output

```
Maximum Bipartite Matching Size (Max Flow): 4

Matched Pairs (Applicant -> Job):
  Applicant_1 <===> Job_1
  Applicant_2 <===> Job_2
  Applicant_3 <===> Job_3
  Applicant_4 <===> Job_4
```

Assignment 6: Flight Schedule Shortest Path (Dijkstra's Algorithm)

Problem Statement

A person wants to fly from city A to city B. Suppose several airlines have connections in different cities, and the flights operate on different schedules. The goal is to minimize travel time or cost. Write a program that implements the shortest-path algorithm to help the person make the most efficient travel arrangements.

Algorithm Design

- Model the flight network as a directed weighted graph $G = (V, E)$, where vertices V represent cities and directed edges (u, v, w) represent scheduled flights with non-negative

edge weight $w \geq 0$ (ticket fare or duration in minutes).

- We use **Dijkstra's Algorithm** utilizing a binary min-heap priority queue (`heapq`) to find the single-source shortest path from city A to city B.
- We maintain a predecessor mapping to reconstruct the complete sequential itinerary (cities and individual flight legs).
- **Time Complexity**: $O((|V| + |E|) \log |V|)$ using min-heap.

Python Implementation

```python
import collections
import heapq

def shortest_flight_path(flights, start_city, end_city):
    """
    Finds the minimum cost or travel time flight itinerary using Dijkstra's algorithm.
    flights: list of (from_city, to_city, weight, airline)
    Returns: (total_weight, path_of_cities, flight_legs)
    """
    adj = collections.defaultdict(list)
    for u, v, weight, airline in flights:
        adj[u].append((v, weight, airline))

    # Priority queue stores: (current_total_weight, current_city, city_path, flight_legs)
    pq = [(0, start_city, [start_city], [])]
    min_dist = {start_city: 0}

    while pq:
        dist, curr, path, legs = heapq.heappop(pq)

        if curr == end_city:
            return dist, path, legs

        if dist > min_dist.get(curr, float('inf')):
            continue

        for neighbor, weight, airline in adj[curr]:
            new_dist = dist + weight
            if new_dist < min_dist.get(neighbor, float('inf')):
                min_dist[neighbor] = new_dist
                new_leg = {"from": curr, "to": neighbor, "weight": weight, "airline": airline}
                heapq.heappush(pq, (new_dist, neighbor, path + [neighbor], legs + [new_leg]))

    return float('inf'), [], []

# Demonstration for Assignment 6
flight_network = [
    ("Surat", "Mumbai", 2500, "Air India"),
    ("Surat", "Ahmedabad", 1200, "SpiceJet"),
    ("Mumbai", "Delhi", 4200, "IndiGo"),
    ("Ahmedabad", "Delhi", 3200, "Vistara"),
```

```
    ("Mumbai", "Bangalore", 3100, "IndiGo"),
    ("Bangalore", "Delhi", 4500, "Air India"),
    ("Ahmedabad", "Jaipur", 1800, "SpiceJet"),
    ("Jaipur", "Delhi", 1500, "IndiGo")
]

start = "Surat"
destination = "Delhi"

min_cost, path, legs = shortest_flight_path(flight_network, start, destination)

print(f"Optimal Flight Route from {start} to {destination}:")
print(f"Route: {' -> '.join(path)}")
print(f"Total Cost: Rs {min_cost}")
print("\nDetailed Flight Itinerary:")
for i, leg in enumerate(legs, 1):
    print(f"  Leg {i}: {leg['from']} -> {leg['to']} via {leg['airline']} (Rs {leg['
    weight']})")
```

Program Execution Output

```
Optimal Flight Route from Surat to Delhi:
Route: Surat -> Ahmedabad -> Delhi
Total Cost: Rs 4400

Detailed Flight Itinerary:
  Leg 1: Surat -> Ahmedabad via SpiceJet (Rs 1200)
  Leg 2: Ahmedabad -> Delhi via Vistara (Rs 3200)
```